BinarySearchTree-master

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Binary Search Tree
Programming Focus: C++ class development (inheritance)

Binary Search Tree

For this computer assignment, implement a derived class (as a template) to represent a binary search tree. Since a binary search tree is a binary tree, implement your binary search tree class inheriting from the base class of the binary tree that you implemented in your previous assignment.

The definition of the derived class of a binary search tree (as a template) is given as follows:

```
template <typename T>
class BinarySearchTree : public BinaryTree<T>
{
public:
    void Insert(const T &x);       // inserts node with value x
    bool Search(const T &x) const; // searches leaf with value x
    bool Remove(const T &x);       // removes leaf with value x
private:
    void _Insert(Node<T> *&, const T &);      // private version of insert
    bool _Search(Node<T> *, const T &) const; // private version of search
    void _Remove(Node<T> *&, const T &);      // private version of remove
    bool _Leaf(Node<T> *node) const;          // checks if node is leaf
};
```

- The `Insert()` function inserts a node with the data value `x` in the tree.
- For the `Search()` function, `x` is the data value of a leaf to be searched for. If the search is successful, the `Search()` function returns **true**; otherwise, it returns **false**.
- The `Remove()` function first calls the `Search()` function to determine the result of the search for a leaf with the data value `x`, and if the search is successful, then it calls `_Remove()`; the private version of the `Remove()` function to remove the corresponding leaf from the tree and returns **true**; otherwise, it returns **false**.
- The `_Leaf()` function simply checks if a node is a leaf.

The private versions of the member functions `_Insert()`, `_Remove()`, and `_Search()` can be implemented as recursive functions, but these functions have an additional argument, which is the root of the tree. The private version of the `_Remove()` function unlike its public version does not return any value.

To test your derived class and its member functions, the source file of a driver program **bst.cc** and its header file **bst.h**, are provided. This program generates a set of random integers in the range [1, N] with N = 100 and inserts them in a vector and in a binary search tree. The program searches for each value of the vector in the binary search tree. If the value can be found as the data value of a leaf, then the leaf is removed from the tree. After removing all the leaves, the size of the tree and the data values in nodes are printed in ascending order by traversing the tree in inorder. This process continues until the tree becomes completely empty.

Put the implementation of your binary search tree class in a separate header file, that you create and add to your repository named **binarysearchtree.h**. To use the `class Node` in your program, include the header file **node.h**, by inserting the statement: `#include "node.h"` at the top of your header file btree.h, and include the header file btree.h in your header file **binarysearchtree.h**. Make it sure that each header file is included only once, so the contents of each header file should be guarded as follows:

```
        // include other header files (if any)
        #ifndef BINARYSEARCHTREE_H_
        #define BINARYSEARCHTREE_H_

        // put statements in header file

        #endif // End of BINARYSEARCHTREE_H_
```

The correct output of this program can be found in file **bst.refout**, which is in the same directory with **bst.cc**.

Note: In public versions of `Insert()`, `Remove()`, and `Search()` functions, you'll get a compile error when you pass the root as an argument to these functions. To eliminate the compile errors, you need to pass the `root` as `this->root`.

Assignment Notes:

- Include any necessary headers and add necessary global constants.
- You are not allowed to use any I/O functions from the C library, such as scanf or printf. Instead, use the I/O functions from the C++ library, such as cin or cout.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (you need to construct and add Makefile) so that the TA only needs to invoke the command make to compile your source file and produce the executable file **bst**. Make sure you use exactly the same file names specified here, i.e. **bst**, in your Makefile, otherwise your **submission will get 0 points**.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.